LLM-POWERED API ORCHESTRATION

Agent Copilot

API 智能体编排系统

> 让一句自然语言，自动变成一串真实可执行、**随时可被人接管**的 API 调用链

- **两阶段检索**

  Milvus 向量粗排召回，gte-rerank 精排收敛，DeepSeek-V3 最终定选

- **八环节主循环**

  任务拆解到结果汇总全链路编排，含缺参补全与注入拦截

- **HITL 状态机**

  真实副作用前可挂起，人可批准 / 拒绝 / 改参，状态持久化

TECH STACK

Flask | MongoDB | Milvus | DeepSeek-V3 | gte-rerank-v2

目录

从「为什么难」到
「怎么解决」再到
「工程上如何可用」

01 **挑战：为什么难** P03 – P04

工具爆炸、参数繁琐、链路复杂、风险不可逆——四堵墙

02 **架构：如何解决** P05 – P09

四层分层、两阶段检索、八环节主循环、HITL 状态机

03 **工程化：真实可用** P10 – P11

同步提交异步轮询，长任务不阻塞，状态可恢复

CHAPTER 01

挑战：为什么难

01

让大模型直接调用真实业务系统时，

会撞上四堵墙

四堵墙

让大模型直接调用真实业务系统时，会撞上四堵墙

工具爆炸

300+

单个业务系统的接口数量

成百上千个 API 散落在接口文档里，名称相近、语义重叠。 人工翻文档定位接口，光是确认「该用哪一个」就要耗掉大量时间； 而选错的代价，是后面所有参数与调用全部作废。

人工方式：~~翻文档→猜接口→试参数→反复试错~~

参数繁琐

必填项、嵌套结构、易错枚举值，手写请求体反复试错。

链路复杂

多步任务需要拆解与上下文传递，一次调用解决不了。

风险失控

不可逆

DELETE / 支付一旦跑偏不可逆，且全程黑盒无法干预。

判断 四堵墙里前三堵可以靠「更强的模型」硬扛，只有**风险失控**无法靠重试兜底——它必须在架构层面被设计进去。

CHAPTER 02

架构：如何解决

四层分工 + 两阶段检索

八环节主循环 + HITL 状态机

P06
系统分层
接入 / 编排 / 能力 / 存储

P07
两阶段检索
粗排 ×2 → 精排 → LLM 定选

P08
规划主循环
八环节 + 回环推进

P09
HITL 状态机
六状态 + 风险分级

LAYERS

系统分层

接入、编排、能力、存储四层分工，各司其职

设计取舍

分层不是为了好看——它让「是否需要人工确认」变成工具表上的数据字段，而不是散落在业务代码里的 if。

四层各承担什么

接入层 Flask API Gateway — JWT 鉴权 · 权限装饰器 · Swagger 文档

编排层 ApiPlanningHub — 任务拆解循环 · 子任务推进 · 结果汇总

能力层 Selection / Param / Task — 工具选择 · 参数抽取与补全 · 注入检测

存储层 Milvus + MongoDB — 向量工具库 · rerank 缓存 · 任务状态持久化

策略数据驱动 是否确认、何种粒度由工具表上的 hitl_required 与 risk_level 字段决定，不写死在业务逻辑里。

两阶段检索

召回要全，精排要准，LLM 只在收敛后的候选里做决策

STAGE 01 · 粗排　Milvus 向量召回 topK × 2 = 10

queryProduct　getDetail　searchGoods　listCategory

getUserProfile　createOrder　sendCoupon　getStock　listBrand

getReview

RERANK　gte-rerank-v2

STAGE 02 · 精排　收敛至 topK = 5，其余淘汰

queryProduct　getDetail　searchGoods　listCategory

getUserProfile　~~5 个淘汰~~

放大系数 ×2 是召回率与延迟的折中——精排只对粗排结果排序，×2 能兜住粗排的语义漂移，又不至于让 rerank 成本翻倍。

STAGE 03 · 定选

DeepSeek-V3 在 5 个候选内决策

queryProductInfo

✓ 命中目标工具

为什么不让 LLM 直接选全部候选？

候选越多，LLM 越容易在长列表里选错。先用向量与重排把无关项物理剔除，等于把「语义判断」和「列表定位」两件事解耦——前者交给模型，后者交给向量库。

规划主循环

单任务直调，多任务进入 while 循环逐个推进子任务

01 **任务拆解** gen_root_task

02 **工具选择** 两阶段召回

03 **参数抽取** extraction_params

04 **缺参补全** supplement

05 **注入检测** gen_judge_task

06 **真实调用** tool_use

07 **循环校验** loop_validate

08 **结果汇总** tool_summary

↻ 子任务未完成 → 取上下文生成下一个子任务描述

注入拦截

gen_judge_task在每次真实调用前判断请求参数是否携带提示注入特征，命中即中止整条链路。

自动安全闸——不依赖人是否记得点确认。

防死循环

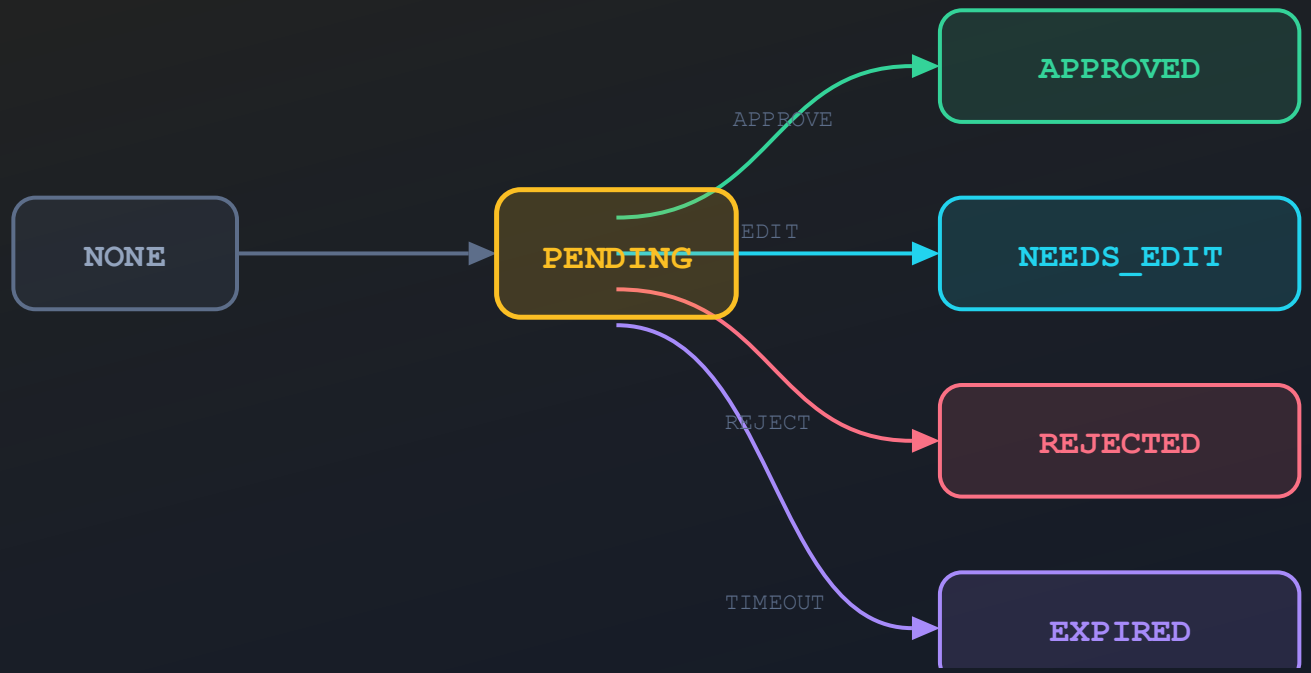
loop_validate检测工具重复调用；一旦判定为循环调用，立即中止任务并给出原因。

缺参补全复用同一套检索链路再走一轮，而不是特设补参逻辑。

可挂起的执行状态机

真实副作用发生前挂起，人可批准 / 拒绝 / 改参

STATE MACHINE · Task.hitl_status



REJECTED / EXPIRED → 任务终止

持久化 pending_action 快照与执行上下文写入 MongoDB，进程重启后可从断点继续

超时 30min → EXPIRED

幂等守卫 subtask_index + hitl_status

RISK POLICY · 混合粒度

low	GET 只读	**单调用级**
medium	POST / PUT / PATCH	**单调用级**
high	DELETE / 支付 / 额度	**子任务级**

为什么 high 要整段确认?

不可逆操作的风险来自组合，不是单次。逐次确认拦不住「三个 DELETE 连起来」这种累积伤害， 所以 high 级选择整段子任务确认。

CHAPTER 03

工程化：真实可用

同步提交、异步轮询

长任务不阻塞，状态可恢复

03

P10 执行模型

POST /api_planning 提交即返回 task_id，长任务在后台线程推进，不阻塞调用方

P11 进度可见

GET /api_task_status 轮询任务状态，running 进度到 done 全程可查

同步执行 · 异步轮询

提交即返回，长任务后台跑，前端轮询进度

task-stream.log

```
› POST /api_planning
› task_id = a7f3... 已提交，线程池接管
›   └ gen_root_task: 多任务 → 进入 while 循环
›   └ 子任务 1: queryProductInfo 调用成功
› GET /api_task_status → running  (2/5)
›   └ 子任务 2: getStock 缺参 → 自动补全
› GET /api_task_status → running  (4/5)
› GET /api_task_status → done     (5/5)
› tool_summary 汇总返回
```

子任务推进进度 5 / 5 步

ThreadPoolExecutor

按 CPU 核数自适应线程池大小

LLM 调用全异步

单次模型调用不阻塞主流程

状态持久化

任务状态写入 MongoDB，可恢复

为什么用轮询而不是 Webhook?

调用方本身就是前端轮询组件。沿用「提交 + 轮询」模型，就不必引入消息队列与回调服务，部署复杂度保持不变。

从「会聊天」
到「会干活」

检索更准 · 编排更稳 · 风险可控

1 检索更准

两阶段召回

先物理剔除再决策

2 编排更稳

八环节主循环

缺参自动补全

3 风险可控

注入拦截 + 人工闸

不可逆操作分级确认

4 状态可恢复

断点持久化

进程重启可续跑

Agent Copilot